



Algorithmen und Datenstrukturen

Leon Johann Brettin



Algorithmen und Datenstrukturen

11. Bäume



B-Bäume

- Idee: Baumhöhe ausgeglichen, aber Verzweigungsgrad variiert
- Ausgangspunkt: ausgeglichener, balancierter Suchbaum
- **Ausgeglichen oder balanciert**: alle Pfade von der Wurzel zu den Blättern des Baumes gleich lang
- **Seiten**:
 - Speichereinheiten, die auf einem externen Medium verwaltet werden können. Deshalb Sprechweise „Seiten“ statt „Knoten“.
 - Initial ist nur eine Seite im Hauptspeicher, Seiten werden nur bei Bedarf nachgeladen
- Datenbankbereich: Knoten der Suchbäume zugeschnitten auf Seitenstruktur des Datenbanksystems
- mehrere Zugriffsattributwerte auf einer Seite
- Ähnlich 2-3-4 Bäume
- **Mehrweg-Bäume**

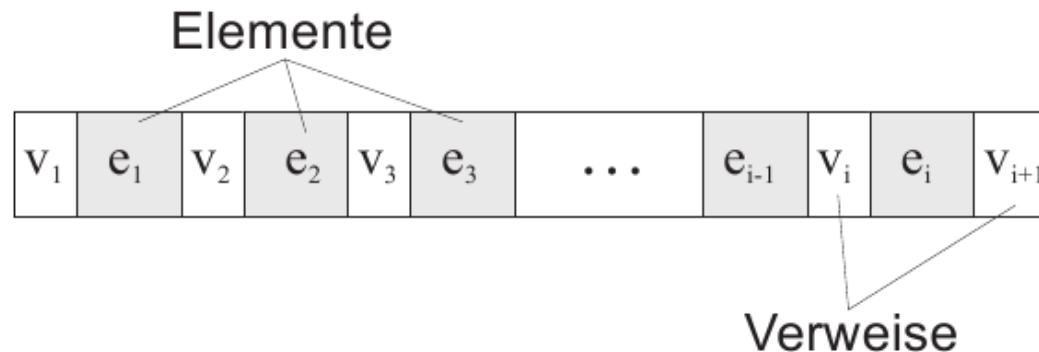


B-Baum

- **B-Baum** von Bayer und McCreight
 - B für balanciert, breit, Bayer, NICHT: binär
- dynamischer, balancierter Suchbaum
- Mehrwegebaum *wäre* völlig ausgeglichen, wenn
 - alle Wege von der Wurzel bis zu den Blättern gleich lang *wären* und
 - jeder Knoten gleich viele Einträge *hätte*.
- Vollständiges Ausgleichen wäre zu aufwendig, deshalb
- B-Baum-Kriterium:
 - Jede Seite außer der Wurzelseite enthält zwischen m und $2m$ Schlüsselwerte
 - Alle Blattseiten liegen auf gleichem Niveau.

Aufbau eines B-Baums

- Der Aufbau einer Seite (eines Knotens) ist dabei der folgende:
 - Jede Seite enthält i geordnete Elemente oder Schlüsselwerte.
 - Es gibt ein m mit $m \leq i \leq 2m$.
 - Dieses m wird als **Ordnung** des B-Baums bezeichnet.
 - Zusätzlich zu den Schlüsselwerten enthält jede Seite noch Verweise auf die Kindknoten mit den Unterbäumen.
 - Für jeden inneren Knoten sind jeweils $i + 1$ Verweise vorhanden.





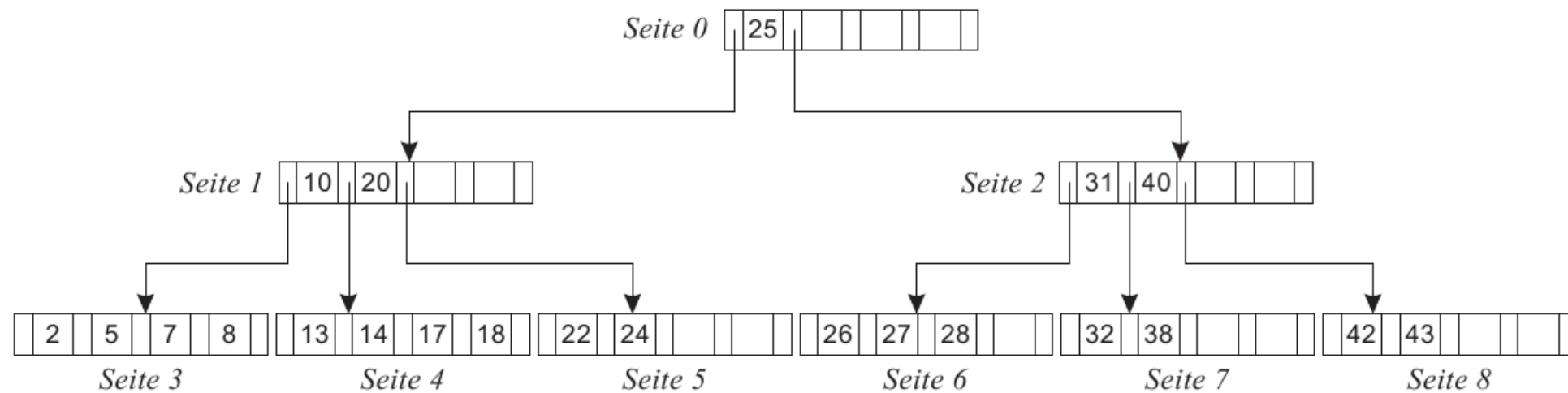
Suche im B-Baum

- Suche ist Verallgemeinerung der Suche auf 2-3-4 Baum
- Suchen beginnend im Wurzelknoten unter den geordneten Elementen nach dem ersten Element, das größer als der gesuchte Schlüssel ist.
 - binäre Suche!
- Steige rekursiv in den entsprechenden Teilbaum ab. Ist der gesuchte Schlüssel größer als alle Elemente eines Knotens, dann steige in den letzten Teilbaum ab.
- In einem Blattknoten, suche, ob der Schlüssel unter den Elementen des Blattes ist, sonst ist der Schlüssel nicht im Knoten und damit im Baum enthalten.



Beispiel: Suche im B-Baum

- Beispiel: Suche nach Schlüssel 38 im B-Baum der Ordnung $m = 2$: Es werden nur die Seiten 0, 2, 7 geladen und dort eine lineare oder binäre Suche durchgeführt.





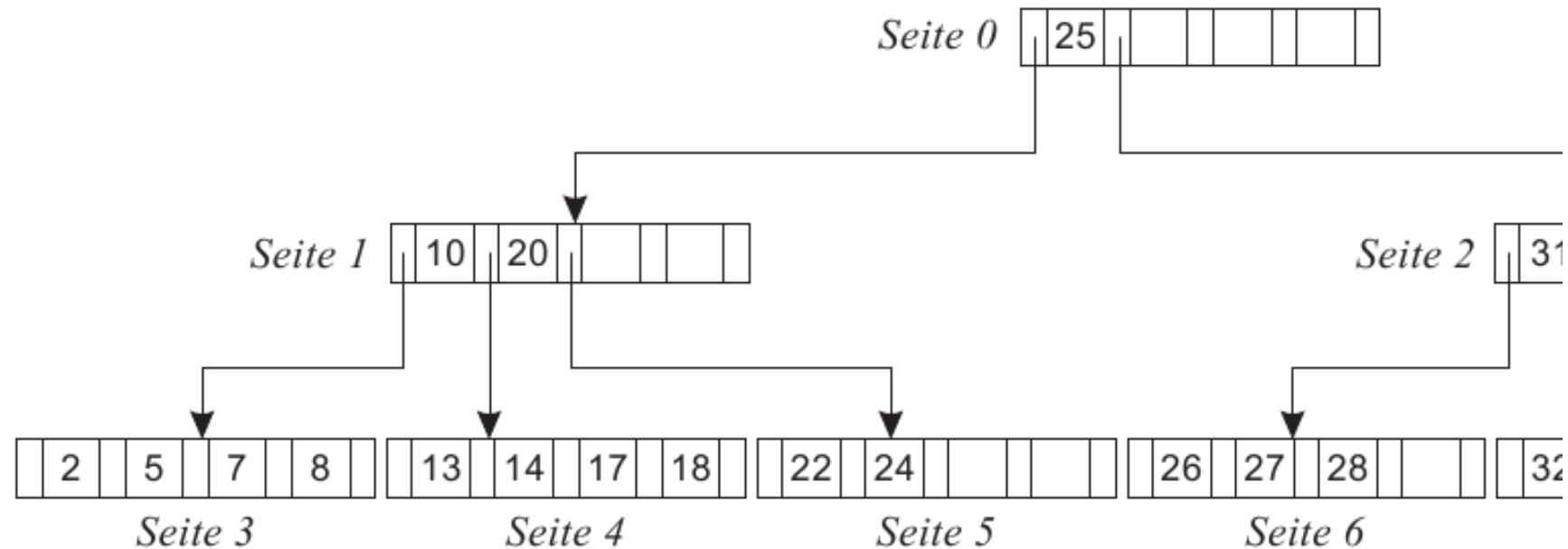
Einfügen in B-Bäumen

- Einfügen eines Wertes w
 - Blattseite suchen
 - passende Seite hat $n < 2m$ Elemente $\rightarrow w$ einsortieren
 - passende Seite hat $n = 2m$ Elemente $\rightarrow$ neue Seite erzeugen
 - erste m Werte auf Originalseite
 - letzte m Werte auf neue Seite
 - mittleres Element in Vaterknoten nach oben
 - eventuell dieser Prozess rekursiv bis zur Wurzel
 - B-Bäume wachsen an der Wurzel!



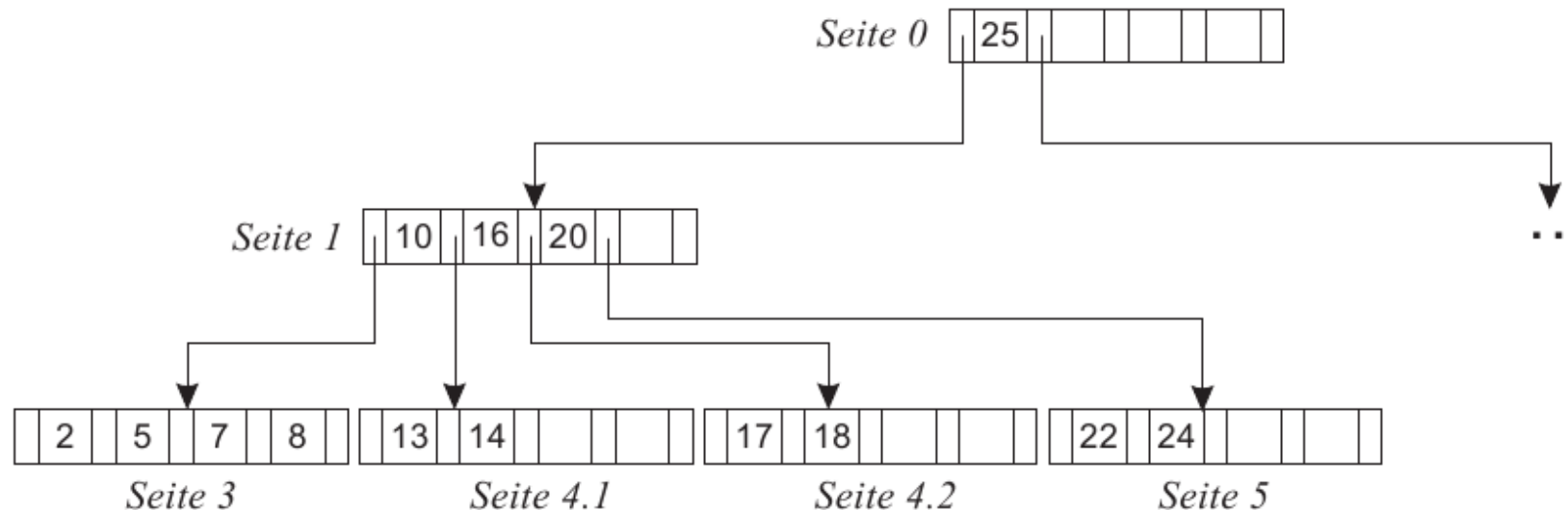
Beispiel: Einfügen in B-Bäumen

- Einfügen von Element 16 in den B-Baum
 - Blattseite suchen



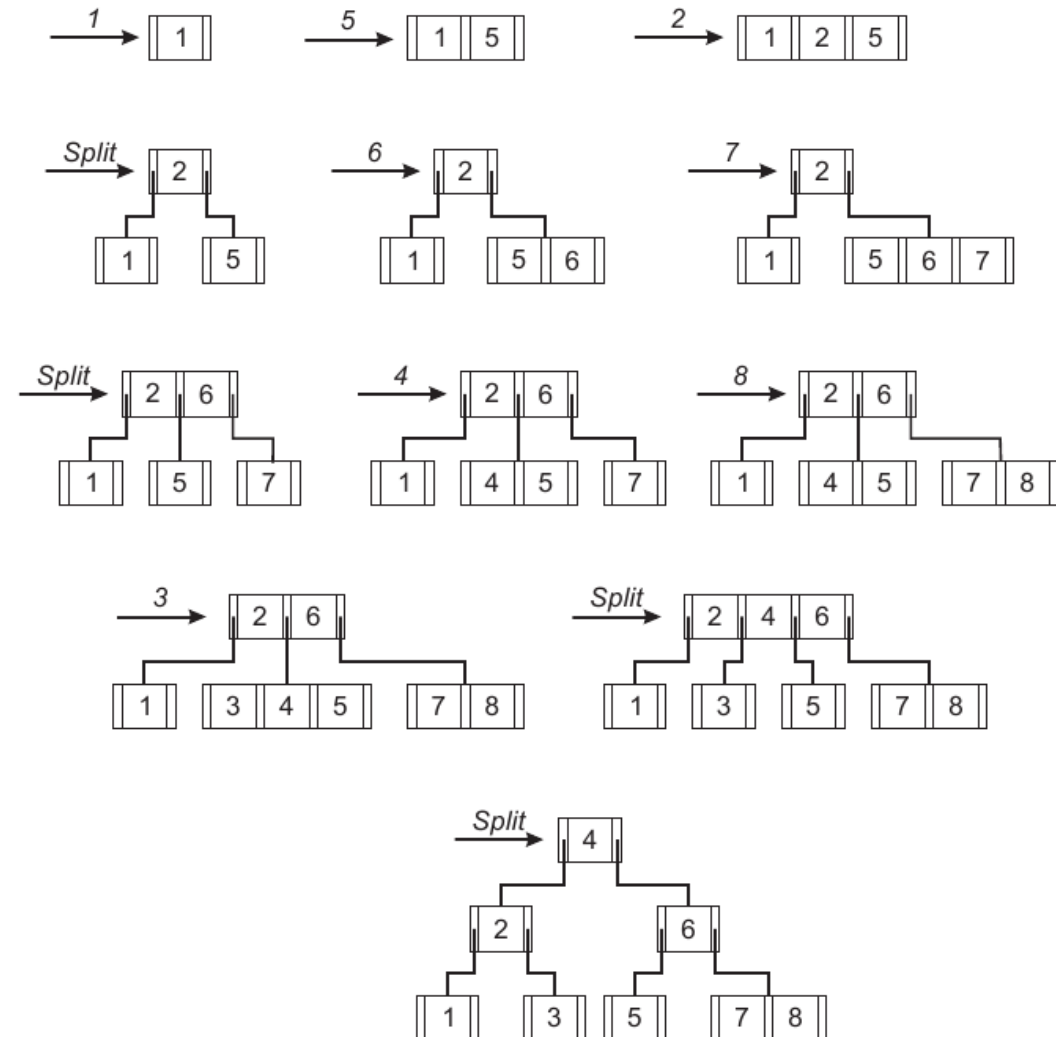
Beispiel: Einfügen in B-Bäumen

- Einfügen von Element 16 in den B-Baum
 - Bei Suche wird passende Seite 4 gefunden. Aber diese enthält bereits 4 Elemente.
 - Neue Seite wird erzeugt und die Elemente aufgeteilt.



Beispiel Aufbau B-Baum mit Einfügeoperationen

- B-Baum der Ordnung $m = 1$. D.h. auf jeder Seite kommen mindestens 1 und höchstens 2 Elemente vor.





Löschen in B-Bäumen

- Problem: bei weniger als m Elementen auf Seite: Unterlauf
 - entsprechende Seite suchen
 - w auf Blattseite gespeichert
 - $\Rightarrow$ Wert löschen, eventuell Unterlauf behandeln
 - w nicht auf Blattseite gespeichert
 - $\Rightarrow$ Wert löschen
 - durch lexikographisch nächstkleineres Element von einer Blattseite ersetzen
 - eventuell auf Blattseite Unterlauf behandeln
- analog Löschen in klassischen Suchbaum!

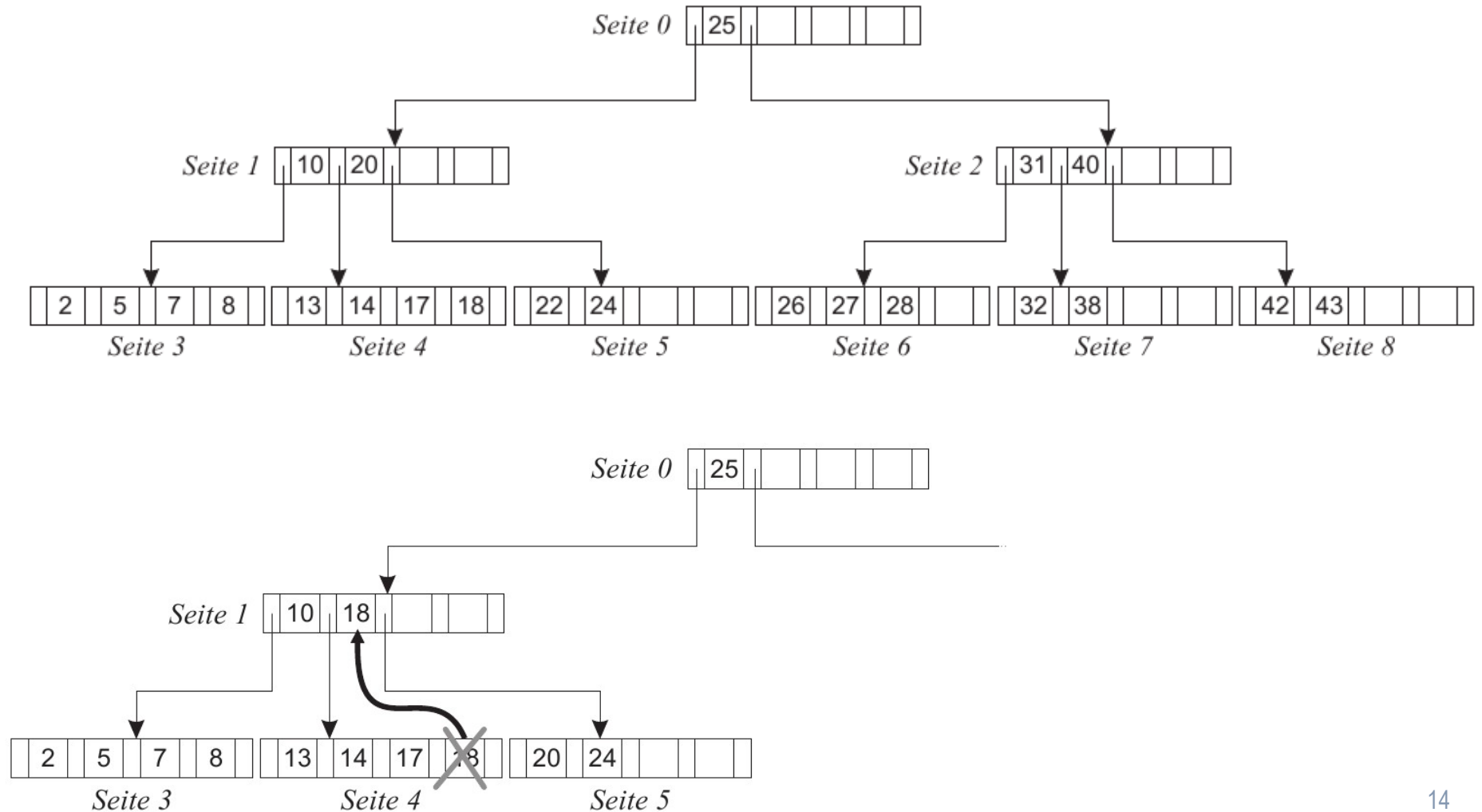


Löschen in B-Bäumen: Unterlauf

- Unterlaufbehandlung
 - Ausgleichen mit einer benachbarten Seite
(benachbarte Seite hat n Elemente mit $n > m$)
 - oder
 - Zusammenlegen zweier Seiten zu einer Seite
(Nachbarseite hat $n = m$ Elemente)
 - das passende „mittlere“ Element vom Vaterknoten dazufügen
 - in Vaterknoten eventuell (rekursiv) Unterlauf behandeln
 - → B-Bäume schrumpfen an der Wurzel!

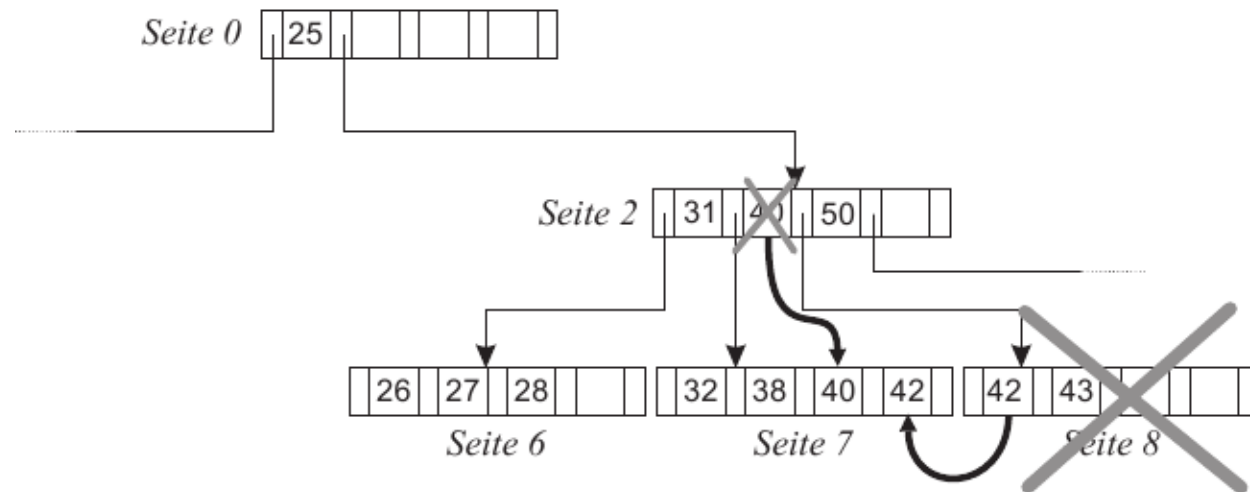
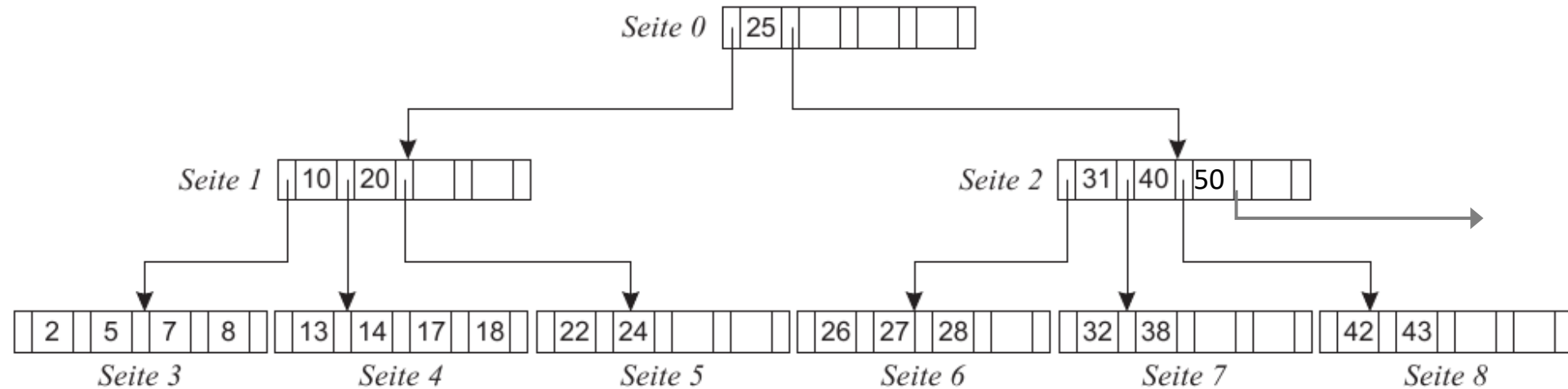


Beispiel: Löschen von 22, Behandlung Unterlauf durch Ausgleichen





Beispiel: Löschen von 43, Behandlung Unterlauf durch Zusammenlegen





Komplexität der Operationen

- Aufwand beim Einfügen, Suchen und Löschen im B-Baum immer $O(\log_m n)$ Operationen
- entspricht genau der „Höhe“ des Baumes
- beliebt für sehr große Datenbestände (mit großer Knotengröße)
- Konkretes Beispiel:
 - Seiten der Größe 4 KB, Zugriffsattributwert 32 Bytes, 8-Byte-Zeiger
 - zwischen 50 und 100 Indexeinträge pro Seite; Ordnung dieses B-Baumes 50
 - 1.000.000 Datensätze: $\log_{50} (1.000.000) = 4$ Seitenzugriffe im schlechtesten Fall
 - optimiert Anzahl von der Festplatte in den Hauptspeicher zu transferierenden Blöcke!



Zusammenfassung Bäume

- Binäre Bäume
- Ausgeglichene Bäume
 - Rot-Schwarz Bäume
 - Suchen, Einfügen, Löschen $O(\log n)$
 - AVL-Bäume
 - Suchen etwas schneller; Einfügen, Löschen etwas langsamer, aber $O(\log n)$
 - B-Bäume
 - Für externe Speicher (Datenbanken)
 - interne Operationen (Umkopieren) schnell.



Welche Eigenschaft haben B-Bäume nicht?

- A. Alle Blätter sind auf einer Ebene
- B. Die Schlüssel sind geordnet
- C. jede Seite hat die gleiche Anzahl Schlüssel
- D. eine innere Seite mit k Schlüsseln hat $k+1$ Verweise
- E. Maximal eine Seite hat weniger als m Schlüssel.

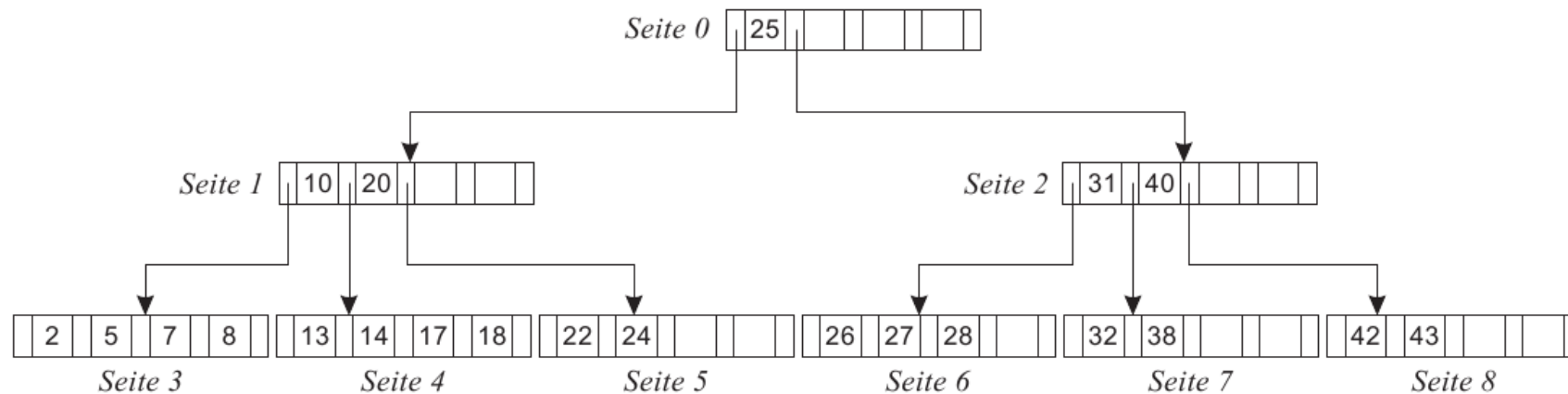


Welche Eigenschaften haben B-Bäume?

- A. Alle Blätter haben m oder mehr Schlüssel.
- B. eine innere Seite mit k Schlüsseln hat $\leq k$ Verweise
- C. jede Seite hat die gleiche Anzahl Schlüssel
- D. jede Seite hat $m \leq k \leq 2 \cdot m$ Schlüssel
- E. Die Wurzel hat mindestens einen Schlüssel

Beispiel: Suche im B-Baum

- Beispiel: Suche nach Schlüssel 23 im B-Baum der Ordnung $m = 2$.
- Welche Schlüssel werden mit dem Suchschlüssel 23 verglichen?



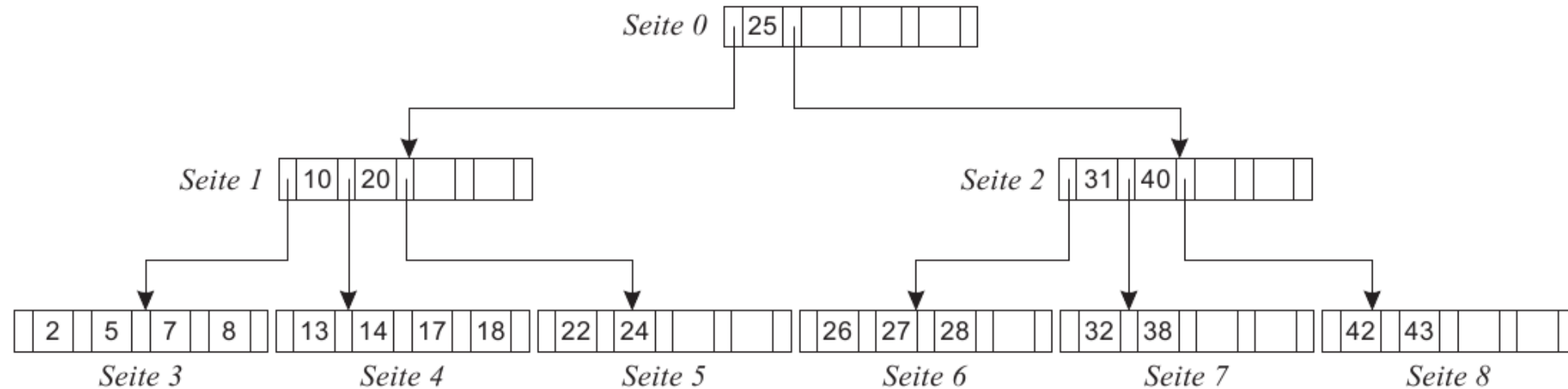


Was dauert am längsten?

1. Der Zugriff auf eine Seite
2. Das Suchen eines Schlüssels in einer Seite
3. Das Umkopieren von Schlüsseln in einer Seite beim Löschen oder Einfügen



Einfügen im B-Baum



Welche Werte können ohne Aufsplitten eingefügt werden?

- A. 0 21 29
- B. 19 21 29
- C. 21 29 39
- D. 23 29 30

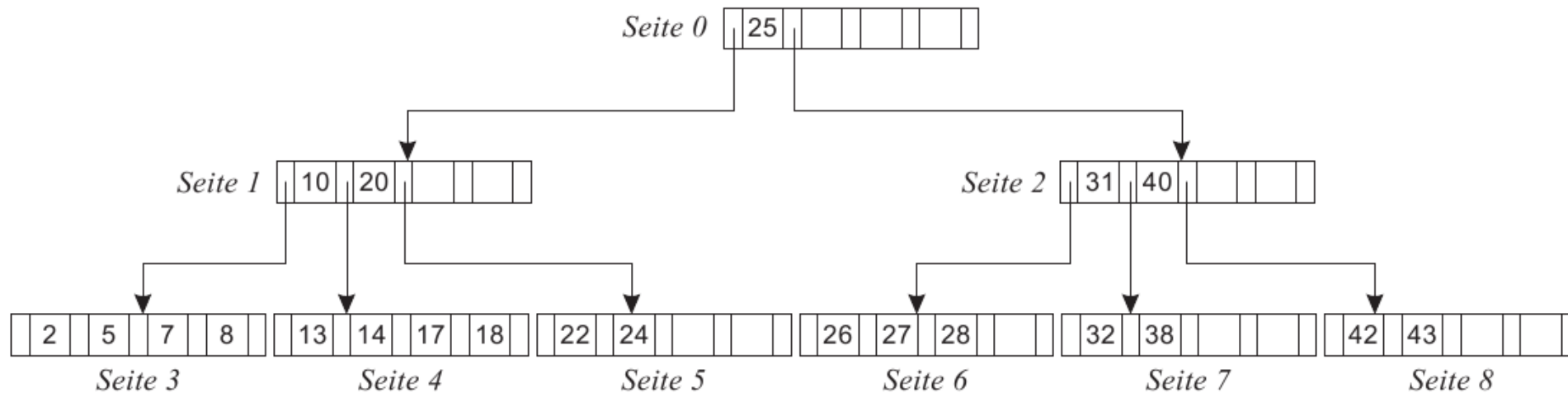


Welche Aussage ist falsch?

- A. Beim Einfügen wird das neue Element immer im Blatt eingefügt.
- B. Beim Einfügen kann es nötig sein, eine Seite aufzuteilen
- C. Beim Einfügen wächst die Höhe des Baumes nur, wenn eine neue Wurzel erzeugt wird.

Beispiel: Einfügen

- Einfügen von Element 0 in den B-Baum





Beispiel: Einfügen

- Einfügen von Element 0 in den B-Baum

